



Logistic Regression

MLAP Lecture Slides

Suresh Manandhar



Introduction to Classification Problems

1. For regression problems we want to fit a known function to given data.
2. In **classification problems** we want learn a **classifier** that predict the correct class given the input data.



The 1-dimensional case

1. Consider the case, we have input data consisting of just one variable
2. e.g. we want to classify people as **young** or **old** depending on **age**.
3. So, input vector $\mathbf{X} = \langle \text{age} \rangle$ and Output $\mathbf{Y} = 0$ (meaning **young**) and 1 (meaning **old**)

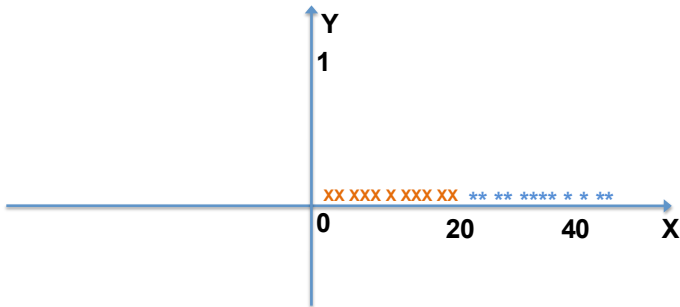


Input data for Age example

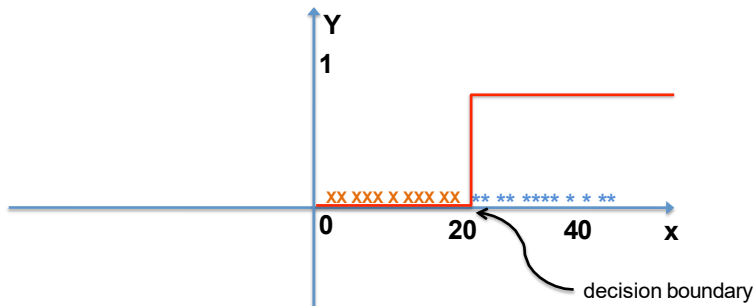
Age	Class
6	0
8	0
10	0
14	0
20	0
21	1
24	1
28	1
30	1



Plotting Age data



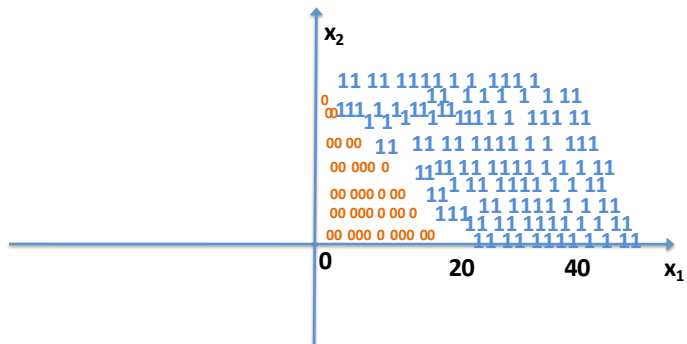
- We want a function that outputs 1 when class is 1 and 0 otherwise



- One solution will be to fit a **step function** at the right point that separates the two classes.
- The transition point is known as the ***decision boundary***.
- The step function gives a hard decision boundary

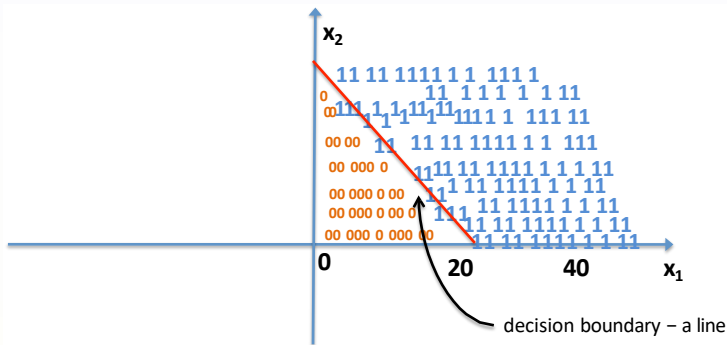


2-dimensional case





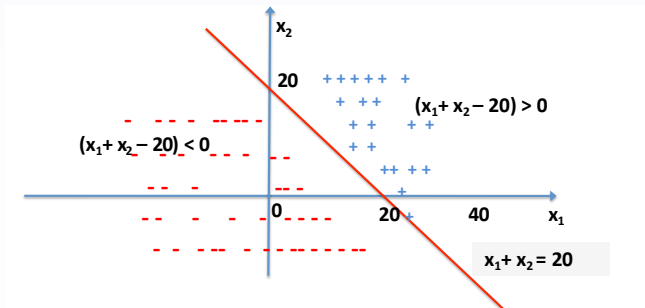
2-dimensional case



- The step function is now in 3-d space, hence only the plan view is shown above.
- The decision boundary is a line instead of a point.



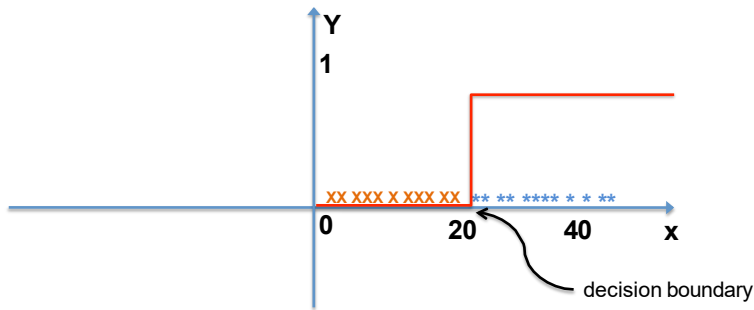
Linear Decision Boundary



- Can be represented as a **line** (in 2-d space)
- Is a **hyperplane** in higher dimensions
- The decision boundary can be represented as a linear equation (e.g. $x_1 + x_2 - 20$)
- Points on the decision boundary return **0**
- **- - -** points return values **< 0**
- **+++** points return values **> 0**



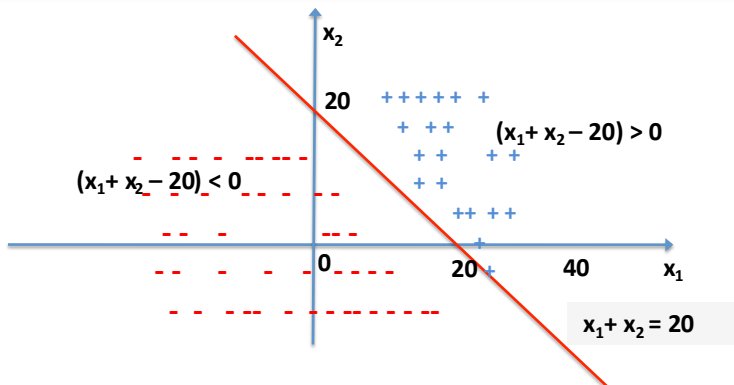
Step function classifier



- Define *step function*:
 $\text{step}(x) = 1$ if $x > 0$ else $\text{step}(x) = 0$
- Then the above example classifier can be represented as:
 $\text{step}(x-20)$



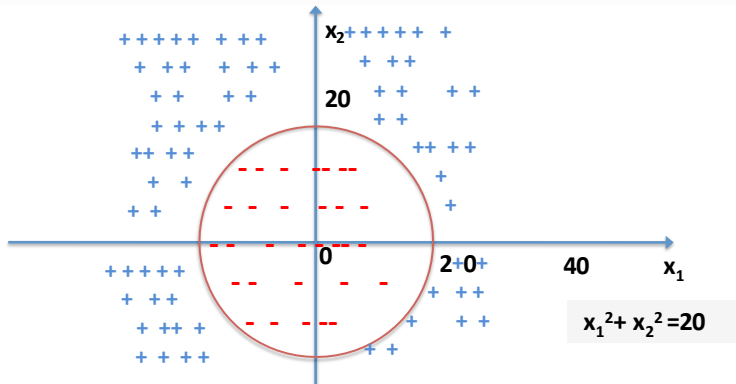
2-D example



- Similarly, the above classifier can be represented as:
 $\text{step}(x_1 + x_2 - 20)$



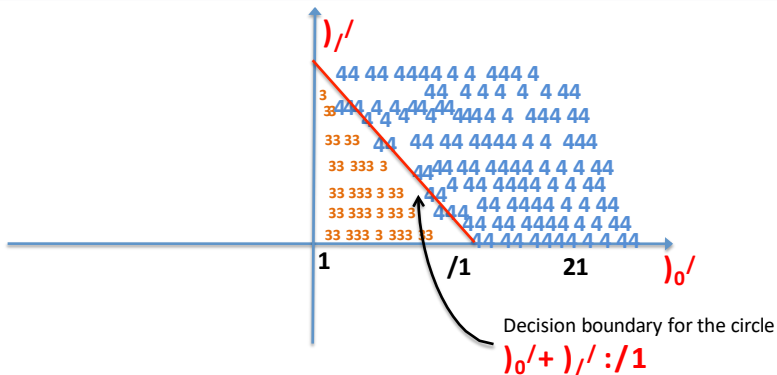
More complex 2-D example



- This classifier can be represented as:
 $\text{step}(x_1^2 + x_2^2 - 20)$

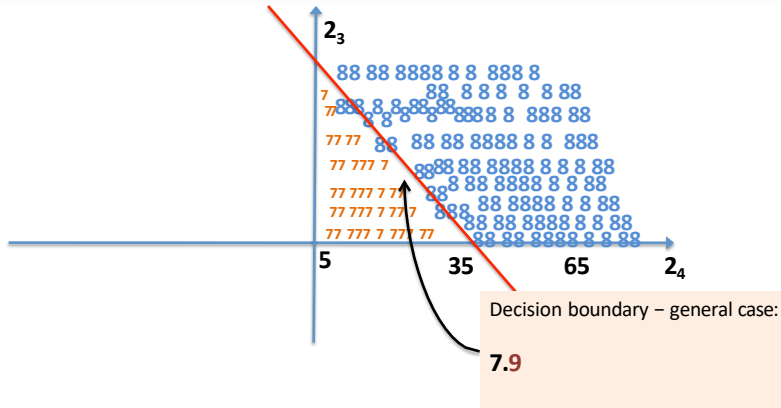


Circle example again



- Viewed in the x_1^2, x_2^2 co-ordinate system, the circle is just a line
- In general, for **linear classification problems**, the decision boundary is a **hyperplane**.

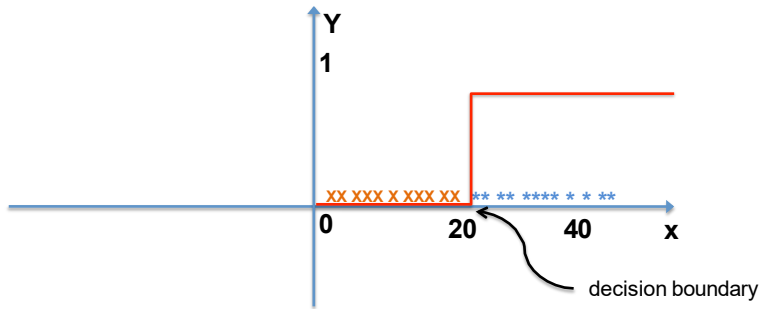
Decision boundary - general case



- Given the input vector $\mathbf{X}$ and the parameter vector θ , the decision hyperplane/boundary can be represented by the dot product $\mathbf{X} \cdot \theta$,



Soft Classifier

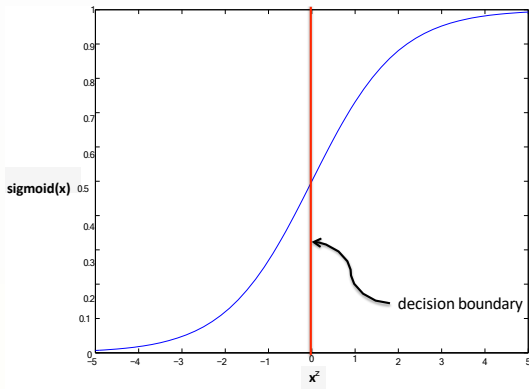


Instead of a hard step classifier we want:

- smooth classifier
- continuously differentiable
- gives values in the 0-1 range (thus can be interpreted as probability)



The sigmoid function



- Forms the basis for logistic regression
- Defined by: $\text{sigmoid}(x) = 1/(1+e^{-x})$



Properties of the sigmoid function

- *symmetric* around **$x = 0$** :
 - ▶ i.e. $\text{sigmoid}(x) = 1 - \text{sigmoid}(-x)$
 - ▶ with $\text{sigmoid}(0) = 0.5$
- $\text{sigmoid}(x) \rightarrow 1$ when $x \rightarrow +\infty$
- $\text{sigmoid}(x) \rightarrow 0$ when $x \rightarrow -\infty$
- replacing, **e** with another constant will still work



Logistic Regression Classifier

- Logistic regression classifier is defined by:

$$p(y = 1 \mid x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

$$p(y = 0 \mid x) = 1 - \frac{1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}} = \frac{1}{1 + e^x}$$

- Thus replacing the step function with a smooth function



Logistic Regression Classifier - n dimensional case

- In general, input variable x will be **M**-dimensional
- Correspondingly, the parameter vector θ is also **M**-dimensional
- x is replaced with the dot product $x\theta$

$$p(y = 1 \mid x, \theta) = \frac{e^{x\theta}}{1 + e^{x\theta}}$$

$$p(y = 0 \mid x, \theta) = \frac{1}{1 + e^{x\theta}}$$



Logistic Regression Classifier - alternative formulation

■ Since:

$$p(y = 1 \mid x\theta) = \frac{e^{x\theta}}{1 + e^{x\theta}} = \frac{e^{x\theta}}{e^0 + e^{x\theta}} = \frac{e^{x\theta}}{e^{x \cdot 0} + e^{x\theta}}$$
$$p(y = 0 \mid x\theta) = \frac{1}{1 + e^{x\theta}} = \frac{e^0}{e^0 + e^{x\theta}} = \frac{e^{x \cdot 0}}{e^{x \cdot 0} + e^{x\theta}}$$

- where $\mathbf{0} = \langle 0, 0, \dots, 0 \rangle$ is the zero **vector**
- thus, $\mathbf{x} \cdot \mathbf{0} = 0$, a **scalar** zero.



- Alternatively, we can use two separate parameters, θ_1, θ_2 , i.e. $\theta = \langle \theta_1, \theta_2 \rangle$, for the two classes:

$$p(y = 0 \mid x, \theta) = \frac{e^{x\theta_1}}{e^{x\theta_1} + e^{x\theta_2}}$$

$$p(y = 1 \mid x, \theta) = \frac{e^{x\theta_2}}{e^{x\theta_1} + e^{x\theta_2}}$$

- It follows that: $p(y = 0 \mid x) = 1 - p(y = 1 \mid x)$ like before.
- The two formulations are equivalent as the representation is **overparameterised**.



Overparameterised representations

- Different values of $\theta = \langle \theta_1, \theta_2 \rangle$ can be equivalent.
- The representation is invariant to addition by a constant vector $C = \langle c, c \rangle$
- $\theta + C = \langle \theta_1, \theta_2 \rangle + \langle c, c \rangle = \langle \theta_1 + c, \theta_2 + c \rangle$
 $p(y = 1 \mid x, \theta + C) =$

$$\begin{aligned} & \frac{e^{x(c+\theta_1)}}{e^{x(c+\theta_1)} + e^{x(c+\theta_2)}} = \frac{e^{xc+x\theta_1}}{e^{xc+x\theta_1} + e^{xc+x\theta_2}} \\ & = \frac{e^{xc} \cdot e^{x\theta_1}}{(e^{xc} \cdot e^{x\theta_1}) + (e^{xc} \cdot e^{x\theta_2})} = \frac{e^{x\theta_1}}{e^{x\theta_1} + e^{x\theta_2}} \\ & = p(y = 1 \mid x, \theta) \end{aligned}$$

- Hence, the representations θ and $\theta + C$ are equivalent.
- Thus the representation is **overparameterised**.



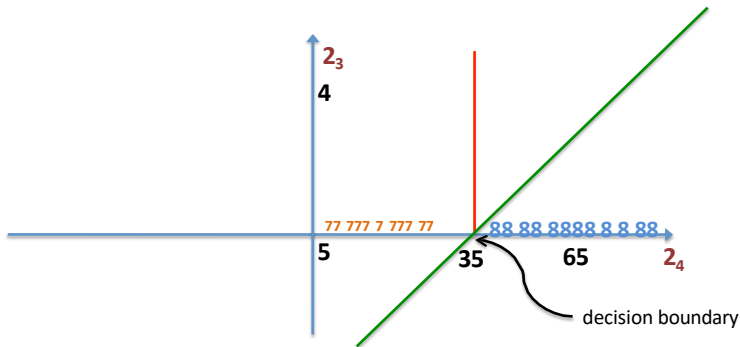
- $\theta = \langle \theta_1, \theta_2 \rangle$ is equivalent to:

$$\begin{aligned}\theta - \langle \theta_2, \theta_2 \rangle &= \langle \theta_1, \theta_2 \rangle - \langle \theta_2, \theta_2 \rangle \\ &= \langle \theta_1 - \theta_2, 0 \rangle\end{aligned}$$

- Thus, there are infinitely many equivalent parameters.
- To avoid searching over redundant/equivalent parameters, one of θ_i is set of 0 .
- Ensures that only a single parameter is the solution.



Equivalent representations



- **Green line** represent parameters $\langle \theta_1, \theta_2 \rangle$ that are equivalent to $\langle 20, 0 \rangle$



MultiClass logistic regression

- When there are multiple classes to be predicted then we need a separate parameter vector for each class.
- Thus θ_1 for class **1**, θ_2 for class **2**, etc.

$$p(y = i|X) = \frac{e^{x\theta_i}}{\sum_j e^{x\theta_j}}$$



Parameter Estimation - The setup

- Given **Data** = *sequence of $\langle \mathbf{X}, \mathbf{y} \rangle$'s*
- $\langle \mathbf{X}, \mathbf{y} \rangle$, a training instance, where $\mathbf{X}$ has **M** *dimensions* and $\mathbf{y}$ is = one of **K** classes.
- $\mathbf{X} = \langle x_{1j} x_{2j} \dots, x_{Mj} \rangle, \mathbf{y} \in \{c_{1j} c_{2j} \dots, c_{Kj}\}$
- We want to train a logistic regression classifier to predict for a given input $\mathbf{X}$ one of the **K** classes.
- Thus we need to learn a parameter vector (actually a matrix):
$$\boldsymbol{\theta} = \langle \boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \dots, \boldsymbol{\theta}_K \rangle$$
where each $\boldsymbol{\theta}_i$ is a **M** dimensional vector.
- $\boldsymbol{\theta}$ defines the parameters for the *decision boundary*.



Maximum Likelihood Estimation (MLE) of Parameters

- The probability of the **Data** given the **Model** is given by:

$$p(\text{Data}|\text{Model}) = \prod_{\langle X, c \rangle \in \text{Data}} p(y = c | X, \theta)$$

assumes data points are independent of each other

- We want to find parameters $\theta_1, \theta_2, \dots$ that maximise $p(\text{Data}|\text{Model})$ i.e.

$$\arg \max_{\theta} p(\text{Data}|\text{Model})$$

- Maximising $p(\text{Data}|\text{Model})$ is same as maximising $\log(p(\text{Data}|\text{Model}))$ - the *log likelihood*



$$\begin{aligned} L(\theta) &= \log(p(\text{Data}|\text{Model})) \\ &= \log \prod_{\langle X, c \rangle \in \text{Data}} p(y = c | X, \theta) \\ &= \sum_{\langle X, c \rangle \in \text{Data}} \log \left(\frac{e^{x\theta_c}}{\sum_j e^{x\theta_j}} \right) \\ &= \sum_{\langle X, c \rangle \in \text{Data}} (x\theta_c - \log(\sum_j e^{x\theta_j})) \\ &= \sum_{\langle X, c \rangle \in \text{Data}} (x\theta_c - \log_sum_exp(x\theta_1, x\theta_2, \dots, x\theta_K)) \\ &= \sum_{\langle X, c \rangle \in \text{Data}} (x\theta_c - \log_sum_exp(x\theta)) \end{aligned}$$



Implementing Gradient descent

- For a given class a we derive the partial derivative for a given dimension i as follows:

$$\begin{aligned}\frac{\partial L(\theta)}{\partial \theta_{a_i}} &= \frac{\partial}{\partial \theta_{a_i}} \sum_{\langle X, c \rangle \in \text{Data}} (X\theta_c - \log(\sum_j e^{X\theta_j})) \\ &= \sum_{\langle X, c \rangle \in \text{Data}} [I(a = c)X_i - \frac{e^{X\theta_a} X_i}{\sum_j e^{X\theta_j}}] \\ &= \sum_{\langle X, c \rangle \in \text{Data}} X_i (I(a = c) - p(y = a | X, \theta))\end{aligned}$$

- Here, $I(a = c)$ is an *indicator function*, defined by:

$$I(a = c) = 1 \text{ if } a = c \text{ else } I(a = c) = 0.$$

- Vectorisation gives us:

$$\frac{\partial L(\theta)}{\partial \theta_a} = \Delta_a L(\theta) = \sum_{\langle X, c \rangle \in \text{Data}} X (I(a = c) - p(y = a | X, \theta))$$



Gradient descent algorithm

- Initialise θ with 0 or random values.
- Read sequence of $\langle X, y \rangle$ (input data)
- Repeat:
 - ▶ $L(\theta) = 0, \Delta L(\theta) = 0$
 - ▶ For each $\langle X, y = c \rangle$:
 - ▶ $L(\theta) += X\theta_c - \log_sum_exp(X\theta)$
 - ▶ For each class a : $\Delta_a L(\theta) += X(I(a = c) - p(y = a|X, \theta))$
 - ▶ Update: $\theta += \text{LearningRate} * \Delta L(\theta)$
- until Termination criteria
- Output θ

- Actually a gradient **ascent** algorithm (since computing argmax)
- Very similar to Linear Regression - structurally identical



Applying Logistic Regression

- Once the logistic regression function has been learnt from input Data, we will have learnt the parameters θ
- To do a **prediction** on new data:
 - ▶ one can compute $p(y = c | X, \theta)$ given by
$$p(y = c | X, \theta) = \frac{e^{X\theta_c}}{\sum_j e^{X\theta_j}}$$
 - ▶ This gives for each class c the probability that class c is *predicted* for input X such that: $\sum_c p(y = c | X, \theta) = 1$
 - ▶ This give us a *sok* classifier that returns a probability distribution over the classes
- If a *hard* classification is desired and we need to choose a single best class, then, simply choose c that gives the highest probability: $c^{hard}(X, \theta) = \arg \max_c p(y = c | X, \theta)$



Softmax regression

- MultiClass logistic regression is also known as **Softmax regression** for the following reason.
- Since e^x grows exponentially fast, small differences between $\mathbf{x}\boldsymbol{\theta}_1$ and $\mathbf{x}\boldsymbol{\theta}_2$ results in large differences in $e^{\mathbf{x}\boldsymbol{\theta}_1}$ and $e^{\mathbf{x}\boldsymbol{\theta}_2}$ (where $\boldsymbol{\theta}_1$ is the parameter for class $\mathbf{c}_1$ and $\boldsymbol{\theta}_2$ is the parameter for class $\mathbf{c}_2$)
- This in turn results in large differences in $p(\mathbf{y} = \mathbf{c}_1 | \mathbf{x}, \boldsymbol{\theta})$ and $p(\mathbf{y} = \mathbf{c}_2 | \mathbf{x}, \boldsymbol{\theta})$
- Thus, in practice, the soft classifier also behaves like a hard classifier (unless, of course, when $e^{\mathbf{x}\boldsymbol{\theta}_1}$ and $e^{\mathbf{x}\boldsymbol{\theta}_2}$ are very close)



Log Sum Exp Trick

- Some computational considerations are needed to implement the **log_sum_exp** function as **exp** grows exponentially and easily results in a floating-point **overflow**
- The following mathematical equivalence can be exploited to avoid this problem:

$$\log \left(\sum_i e^{x_i} \right) = \log \left(\frac{e^c}{e^c} \sum_i e^{x_i} \right) = \log(e^c) + \log \left(\sum_i \frac{e^{x_i}}{e^c} \right) = c + \log \left(\sum_i e^{x_i - c} \right)$$

- To avoid overflow, we choose the constant c to be the maximum possible value of x_i : $c = \max_i x_i$
- Instead of computing the overflow causing expression, we instead compute the safe expression: $c + \log \left(\sum_i e^{x_i - c} \right)$



Log Sum Exp Python code

```
def log_sum_exp(Vector):  
    # first find max of the sequence  
    Max = Vector[0]  
    for LP in Vector:  
        if (LP > Max): Max = LP  
  
    Sum = 0.0  
  
    for LP in Vector:  
        Sum += math.exp(LP - Max)  
  
    return(Max + math.log(Sum))
```



Learning outcomes summary

- Understand the difference between regression and classification
- Understand why decision boundaries are linear and why they correspond to hyperplanes
- Why a sigmoid function is employed. Its properties.
- Formulation of 2-class and K-class logistic regression
- Why the logistic regression formulation is overparameterised.
- Understand MLE parameter estimation for logistic regression.
- Derive the equations for the gradient. Implement, gradient based algorithm.
- Understand why naive implementation of $\log_sum_exp$ is problematic. How to fix this.